Relatório de desenvolvimento das atividades

Instituto de Computação - Universidade Estadual de Campinas

Allan M. de Souza, Rafael O. Jarczewski

Nome: Randerson Araújo de Lemos
RA: 103897
Ferramentas e bibliotecas utilizadas: scapy (sniff, sendp, get_if_hwaddr, getmacbyip, Ether, IP, TCP, UDP, RAW); time; sys; tcpdump; wireshark.
Arquivos alterados: docker-compose.yml (alterado); roteador.py (alterado); firewall/layer_network.py (novo); firewall/layer_transport.py (novo); firewall/rate_limiter.py (novo); firewall/layer_application.py (novo); forward/world2server.py (novo); forward/server2world.py (novo); package/content.py (novo).

Análise Estatística do Tráfego (Sniffer)

Descreva brevemente como o sniffer foi construído. Quais funções do Scapy foram fundamentais (ex: `sniff()`, `wrpcap()`) e como você estruturou o loop de captura para não perder pacotes em alta vazão.

CONSTRUÇÃO NO SNIFFER

O sniffer foi implementado em Python como um roteador de software (*gateway* transparente) entre duas interfaces (servidor - eth0 e clientes - eth1). Ele intercepta o tráfego e submete cada pacote a um *pipeline* de inspeção (L3, L4, L7 e *rate limiting*) na função roteamento(). Após a validação, o script reconstrói os quadros Ethernet (usando cache ARP próprio para MACs), corrige os *checksums* (IP/TCP/UDP) e injeta o pacote na interface de destino.

FUNÇÕES FUNDAMENTAIS DO SCAPY UTILIZADAS

- `sniff()`: Função responsável por manter o ciclo contínuo de interceptação assíncrona dos pacotes diretamente nas interfaces de rede especificadas.
- `sendp()`: Função utilizada para o encaminhamento do tráfego. Visto que o *script* opera como um roteador customizado, o `sendp()` possibilita a injeção direta do pacote na Camada 2 (*Data Link*), bypassando as regras de roteamento padrão do sistema operacional.
- `get_if_hwaddr()` e `getmacbyip()`: Funções necessárias para a obtenção dos endereços MAC físicos do próprio roteador.

ESTRUTURAÇÃO DO LOOP PARA ALTA VAZÃO

Processar pacotes no *user-space* com Python (via Scapy) gera um *overhead* natural. Para garantir que o sniffer suportasse tráfego de alta vazão (inundações e ataques) sem perder

pacotes ou travar o sistema, duas estratégias críticas foram implementadas na chamada do laço principal:

1. **Descarte Imediato de Memória (`store=0`):** Configurar `store=0` força o Scapy a descartar pacotes da RAM após o processamento (`prn=roteamento`), prevenindo esgotamento de memória (OOM) em ataques de negação de serviço.
2. **Filtros BPF no Kernel:** O uso de `filter="net 10.0.0.0/16"` delega a filtragem de pacotes irrelevantes ao *Kernel* (BPF), economizando ciclos de CPU ao evitar a cópia desnecessária de dados para o ambiente Python.

O foco é a caracterização do tráfego coletado sob condições normais de uso.

1) **Filtros e Captura**
   a) **Filtros BPF (Berkeley Packet Filter) utilizados:**
   b) **Justificativa: Por que esses filtros foram escolhidos para a análise?**

R.1)

Para realizar a caracterização do tráfego sob condições normais de uso, a captura dos pacotes foi executada diretamente no roteador utilizando a ferramenta de linha de comando `tcpdump`. O comando exato utilizado para gerar o arquivo de análise (`normal.pcap`) foi:

```
sudo tcpdump -i any net 10.0.0.0/16 -w normal.pcap
```

R.1.a)

O filtro BPF utilizado foi: `net 10.0.0.0/16`.

R.1.b)

Este filtro foi escolhido por dois motivos técnicos principais: escopo de rede e performance.

Em relação ao **escopo**, a notação CIDR `/16` (que trava os dois primeiros octetos do IP) garante que todo o tráfego da infraestrutura do laboratório seja capturado simultaneamente. Isso abrange perfeitamente tanto o servidor (alocado na sub-rede `10.0.1.x`) quanto as máquinas clientes (alocadas na sub-rede dinâmica `10.0.2.x`), permitindo uma visão global e não fragmentada da comunicação. A escolha da interface virtual `any` acoplada a este filtro assegura que o tráfego lateral e transversal seja totalmente registrado.

Em relação à **performance**, a utilização da sintaxe BPF é essencial para a eficiência do *sniffer* (seja via `tcpdump` ou na implementação em Scapy). Ao declarar o filtro, o descarte de pacotes irrelevantes — como ruídos de *broadcast* ou tráfego de gerência fora da faixa `10.0.x.x` — é delegado diretamente ao *Kernel* do sistema operacional. Isso evita cópias desnecessárias de memória para o *user-space* (espaço de usuário), economizando ciclos de CPU e garantindo que o sistema consiga registrar o tráfego de interesse sem perder pacotes (*packet loss*) sob condições de alta vazão.

2) Vazão por Serviço (BPS)
   a) Gráfico
   b) Análise Descritiva

R.2

R.2.a

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDUs
Frame	100.0	3322	100.0	484000	57 k	0	0	0	3322
Linux cooked-mode capture	100.0	3322	13.7	66440	7,870	0	0	0	3322
Internet Protocol Version 4	100.0	3322	13.7	66440	7,870	0	0	0	3322
Transmission Control Protocol	100.0	3322	72.5	351120	41 k	2474	202920	24 k	3322
Telnet	3.9	128	0.2	1144	135	128	1144	135	128
MySQL Protocol	16.9	560	11.7	56480	6,690	560	17840	2,113	1760
Hypertext Transfer Protocol	4.8	160	13.1	63440	7,515	80	11120	1,317	160

Figura 1: ***Printscreen*** **de tabela com informação de BPS gerada pelo Wireshark.**

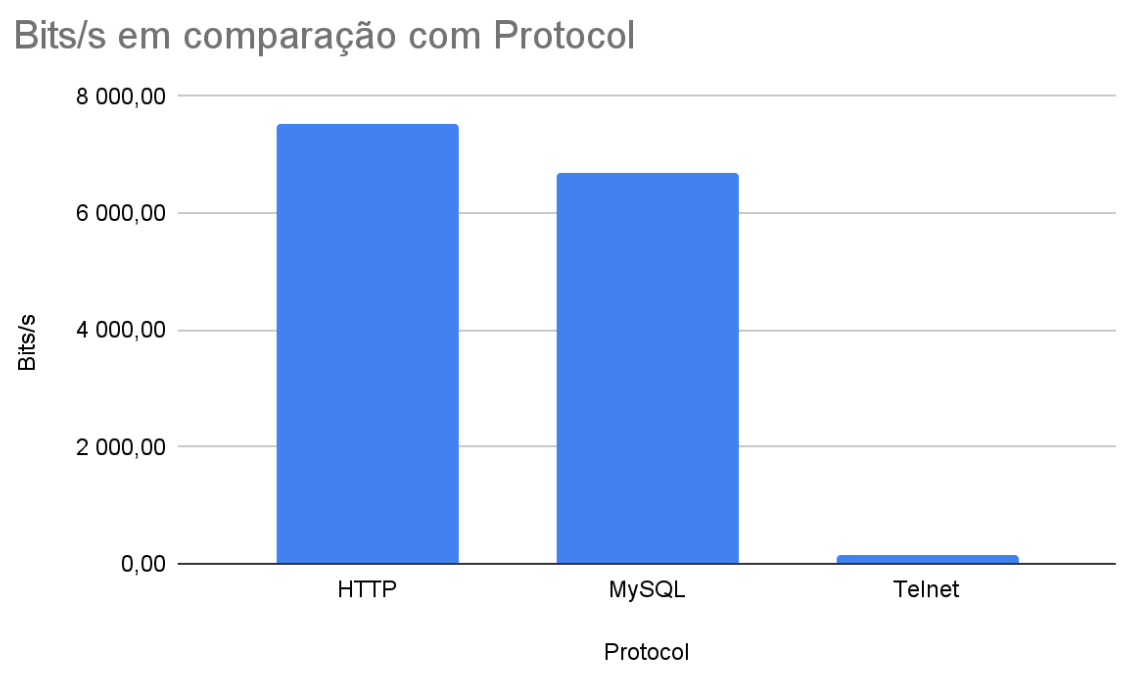


Figura 2: Gráfico em barras de BPS por serviço.

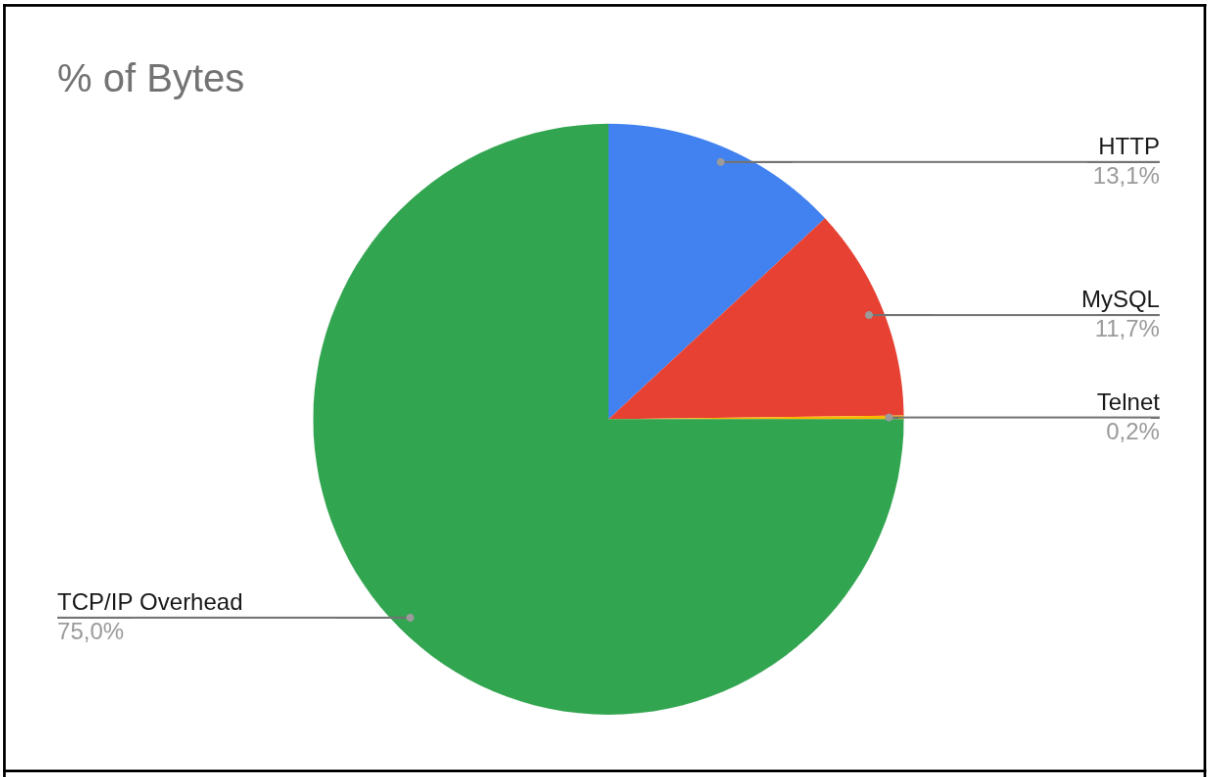


Figura 3: Gráfico de pizza com as porcentagens do volume total de bytes transmitidos por serviços.

R.2.b

A análise da hierarquia de protocolos demonstra que, sob as condições normais de uso do laboratório, a vazão de dados na camada de aplicação foi majoritariamente dominada por requisições web e tráfego de banco de dados. O protocolo **HTTP** apresentou o maior consumo de banda com 7.515 bits/s (representando 13,1% do volume total de bytes transmitidos na captura), seguido estritamente pelo protocolo **MySQL**, com 6.690 bits/s (11,7%).

Observa-se também a presença residual de conexões de gerenciamento via **Telnet** (135 bits/s), que consumiram uma parcela ínfima da capacidade da rede (0,2%). O restante do volume de dados capturado pelo sniffer refere-se estritamente ao *overhead* (cabeçalhos) do protocolo TCP e IP, bem como aos pacotes de controle (como confirmações ACK sem *payload*), fundamentais para a manutenção do estado das conexões HTTP e MySQL ativas entre os clientes e o servidor.

3) **Frequência de Pacotes.**
   a) **Gráfico**
   b) **Análise Descritiva**

R.3
R.3.a

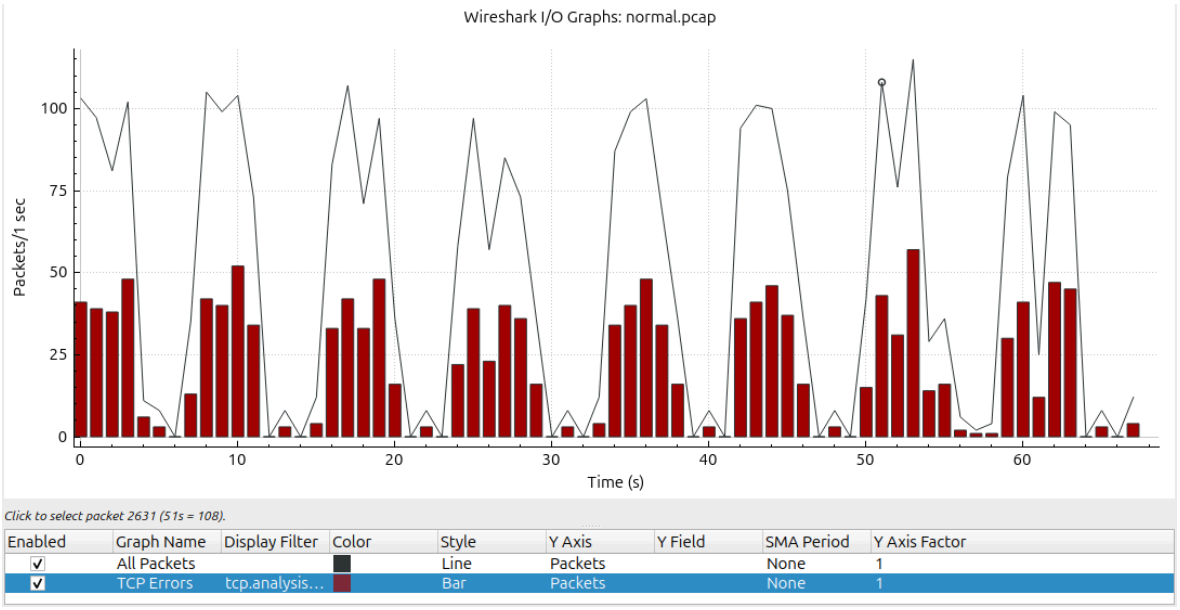


Figura 4: *Printscreen* da frequência de pacotes em formato gráfico.

R.3.b

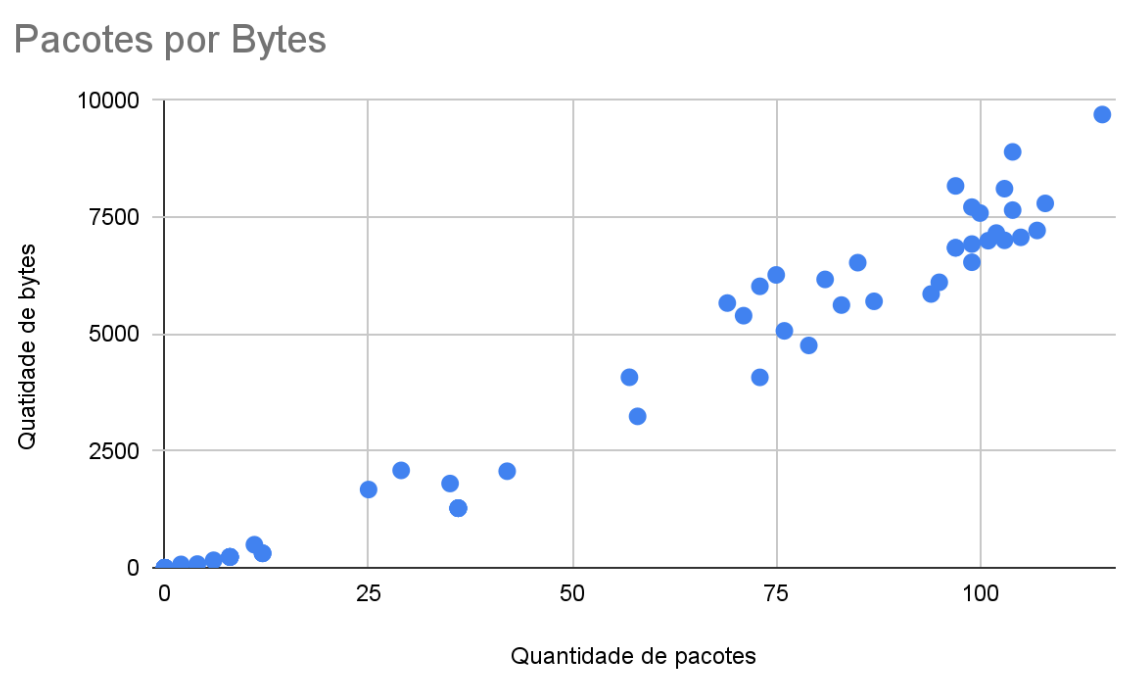
O gráfico de E/S (I/O Graph) revela um padrão de tráfego intermitente e "em rajadas" (*bursty traffic*), típico de uma comunicação cliente-servidor onde requisições são feitas periodicamente. Observa-se que a rede alterna entre períodos de alta atividade, com picos de frequência atingindo aproximadamente **115 pacotes por segundo**, e períodos de latência onde o tráfego cai para valores próximos de zero.

Um ponto relevante na análise é a presença de eventos de controle TCP (identificados pelas barras vermelhas), que mantêm uma média de **40 a 50 pacotes por segundo** durante os picos. Esses pacotes representam retransmissões, confirmações duplicadas ou ajustes de janela, o que é um comportamento esperado em ambientes virtualizados onde o processamento de pacotes via software (como o roteador Scapy desenvolvido) pode introduzir pequenas variações de tempo (*jitter*). A estabilidade dos picos em torno de 115 PPS confirma que o ambiente de rede está operando dentro dos limites de vazão planejados para o laboratório, sem sinais de saturação ou ataques volumétricos.

4) **Volume de pacotes pelo volume de bytes**
    a) **Gráfico de dispersão (Scatter plot).**
    b) **Análise descritiva**

R.4

R.4.a



Figrua 5: Gráfico de dispersão de pacotes por bytes.

R.4.b

A Figura 5 apresenta o gráfico de dispersão entre a frequência de pacotes (eixo X) e o volume de bytes (eixo Y), evidenciando uma forte correlação linear positiva. Esta inclinação constante indica que, independentemente da carga momentânea da rede, o tamanho médio dos pacotes manteve-se estável ao longo de toda a captura, situando-se na faixa restrita dos 70 a 80 bytes por unidade.

Visualmente, os dados dividem-se em dois blocos comportamentais: um agrupamento de baixa atividade (até 40 pacotes/s), que concentra o tráfego residual e de sinalização TCP, e um agrupamento de alta atividade (entre 75 e 115 pacotes/s), que espelha os picos de requisições gerados pelas aplicações HTTP e MySQL.

A ausência de pontos com alta volumetria de bytes e baixa contagem de pacotes descarta a ocorrência de transferências de ficheiros pesados. O gráfico consolida a conclusão de que a comunicação no laboratório consistiu estritamente em transações curtas, com a capacidade da rede a ser amplamente consumida pelo *overhead* de controlo e pelas retransmissões induzidas pelo roteador Scapy.

Detecção de Anomalias e Processamento de Firewall

Nesta etapa, o sniffer atua como um elemento de rede (roteador/firewall) que decide o destino do pacote com base no seu comportamento.

2.1 Comparativo Visual: Fluxo Normal vs. Anômalo

Apresenta a comparação lado a lado de como as métricas se comportam quando um ataque é injetado.

R.2.1

Vamos apresentar o volume de pacotes e de bytes ao longo do tempo aferidos no servidor nas condições de fluxo normal de requisições sem *firewall*, fluxo com ataques de requisições sem *firewall,* e fluxo com ataques de requisições com *firewall.*

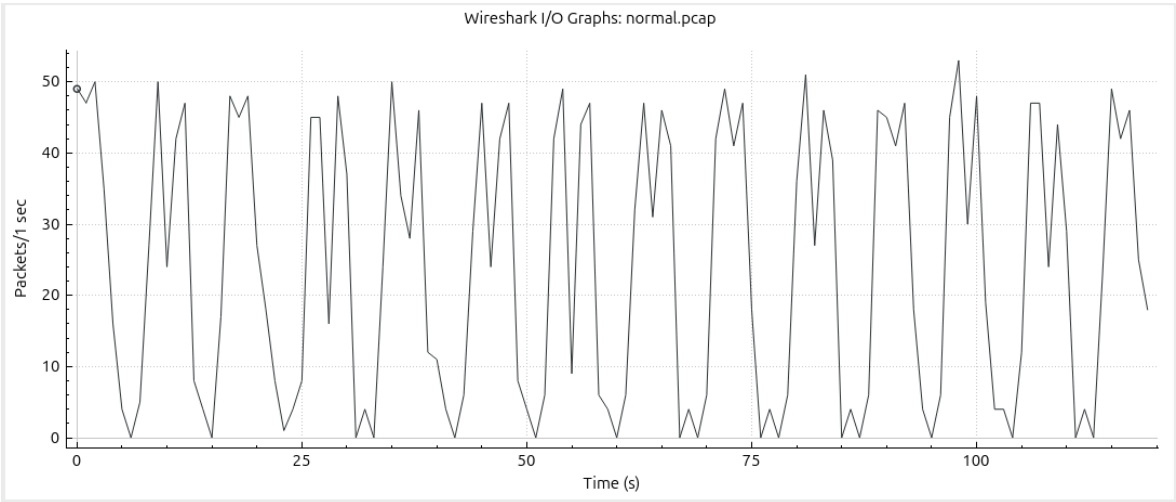


Figura 6: Volume de pacotes ao longo do tempo para fluxo de requisições normal.

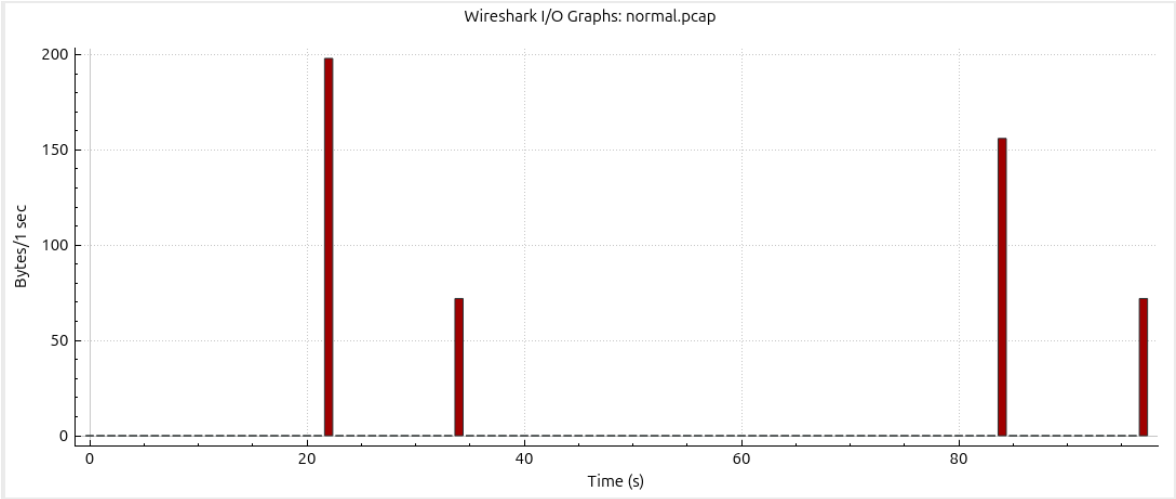


Figura 7: Volume de bytes ao longo do tempo para fluxo de requisições normal.

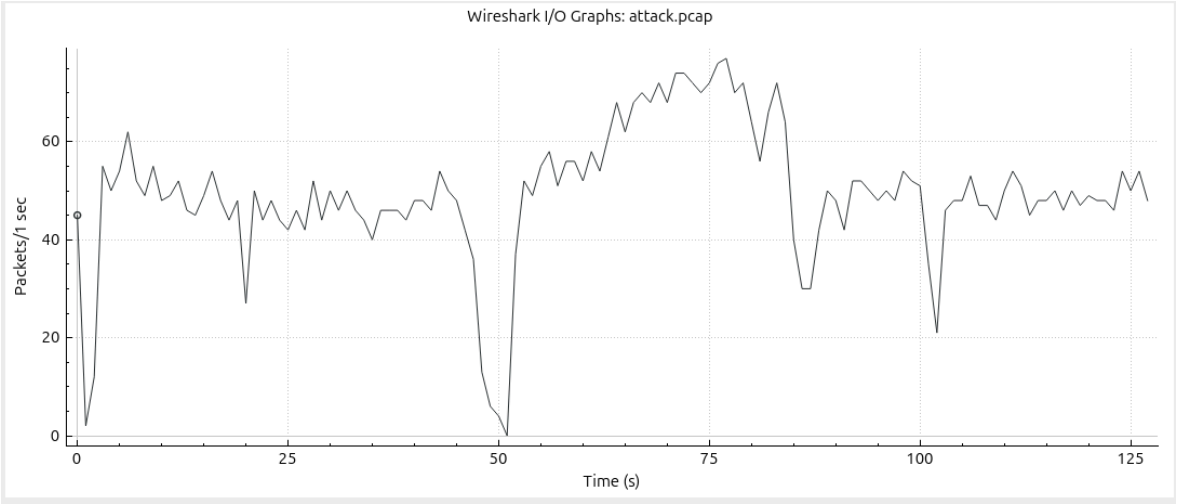


Figura 8: Volume de pacotes ao longo do tempo para fluxo de requisições com ataques.

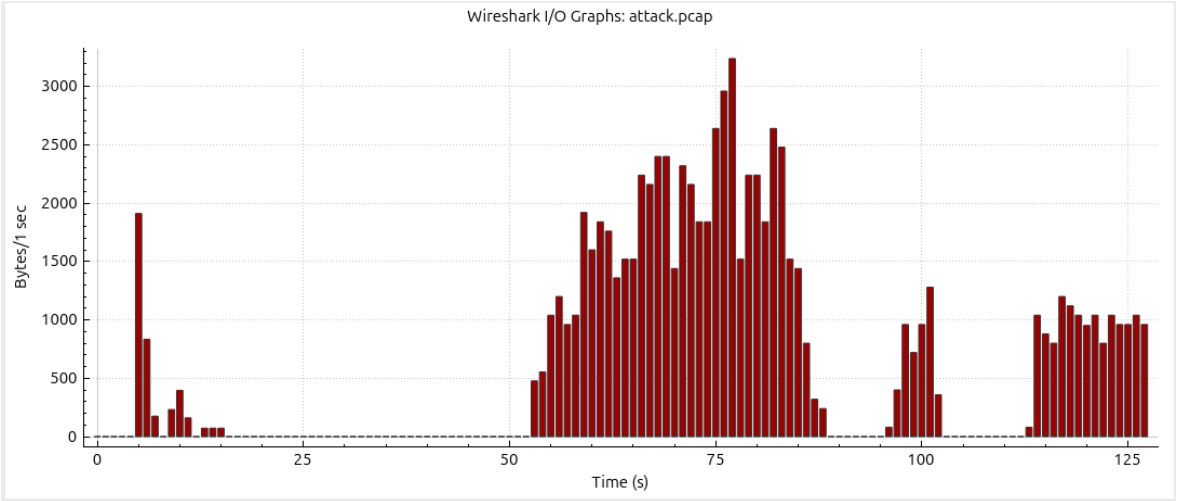


Figura 9: Volume de bytes ao longo do tempo para fluxo de requisições com ataques.

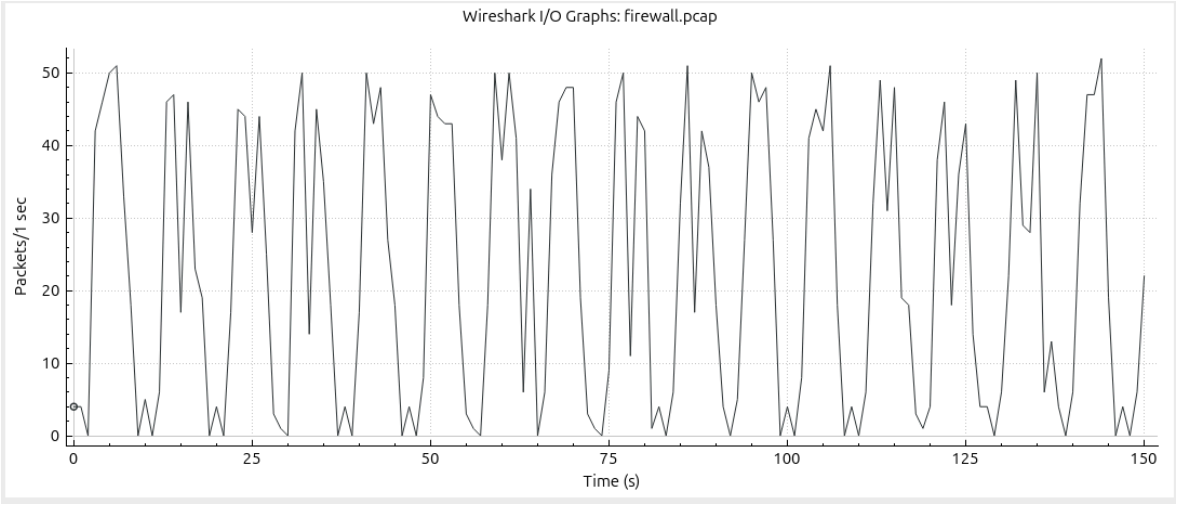


Figura 10: Volume de pacotes ao longo do tempo para fluxo de requisições com ataques e firewall.

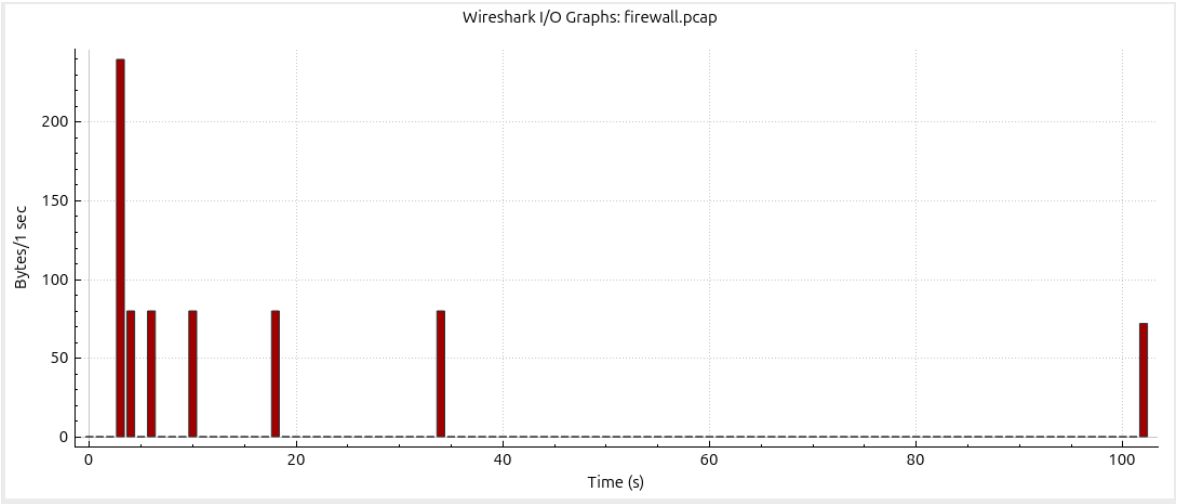


Figura 11: Volume de bytes ao longo do tempo para fluxo de requisições com ataques e firewall.

Com base nas imagens fornecidas, a análise comparativa das métricas de rede (pacotes e bytes por segundo) revela claramente as diferentes fases do ambiente: a linha de base, o impacto da anomalia e a eficácia da mitigação.

Em condições normais (`normal.pcap`) (Figura 6 e Figura 7), a rede apresenta um comportamento altamente periódico e previsível. O gráfico de pacotes por segundo exibe oscilações regulares e consistentes, mantendo-se geralmente entre 0 e 50 pacotes/s, o que sugere trocas de mensagens de sinalização, telemetria ou *keep-alives*. O volume de dados associado a esse tráfego é extremamente baixo e esparso. O gráfico de bytes por segundo mostra apenas picos pontuais que não ultrapassam 200 bytes/s, indicando que a operação padrão da rede não envolve grandes transferências de *payload*.

Quando o ataque é injetado (`attack.pcap`) (Figura 8 e Figura 9), o perfil de tráfego sofre uma distorção severa, caracterizando uma clara anomalia volumétrica. A taxa de pacotes por segundo aumenta e perde sua periodicidade, formando um platô que atinge picos próximos a 80 pacotes/s. Contudo, o impacto mais agressivo é visto na largura de banda: o gráfico de bytes por segundo revela um fluxo de dados intenso, contínuo e volumoso entre os segundos 50 e 90 (e retornando após o segundo 110), chegando a ultrapassar a marca de 3000 bytes/s. Esse salto em relação ao tráfego normal é típico de ataques que buscam exaustão de recursos, como *flooding* (inundação) ou transferência não autorizada de dados pesados.

Com a atuação do elemento de rede atuando como filtro (`firewall.pcap`) (Figura 10 e Figura 11), observa-se o sucesso na contenção da ameaça. O gráfico de pacotes por segundo retorna ao seu padrão original, exibindo novamente as oscilações controladas abaixo de 50 pacotes/s, o que indica que os pacotes anômalos estão sendo ativamente descartados pelo roteador/firewall. A confirmação visual da mitigação é sustentada pelo gráfico de bytes por segundo, onde o volume massivo gerado pelo ataque desaparece por completo. O tráfego de bytes é achatado para níveis mínimos (com picos esparsos não passando de 250 bytes/s), provando que a política de segurança foi capaz de isolar o fluxo malicioso e proteger o destino, restaurando a rede ao seu estado normal de operação.

2.2 Método de Detecção Implementado

Descreva a lógica utilizada para identificar o tráfego inválido.

- **Critério de Decisão: Você não pode utilizar IP ou Porta de destino e origem para isso**
- **Fluxo de Execução:**
  1. **O pacote é capturado pela camada de processamento.**
  2. **O método verifica os atributos em relação à lista negra ou limiares (thresholds).**
  3. **Encaminhamento: Se normal, o pacote segue para `send()`.**
  4. **Drop: Se anômalo, a função retorna um log de alerta e descarta o pacote (não executa o send).**

R.2.2

Para proteger a infraestrutura contra o tráfego inválido, a detecção não se baseia no bloqueio estático de endereços IP ou portas, mas sim em uma arquitetura que olha para diferentes níveis hierárquicos dos pacotes. O *sniffer* atua como uma esteira de avaliação em cascata (*pipeline*), inspecionando o pacote desde as camadas inferiores até a camada de aplicação. Se o pacote falhar em qualquer etapa, ele é imediatamente descartado, poupando recursos computacionais.

O fluxo de processamento e as estratégias adotadas dividem-se nas seguintes etapas:

- **Camada de Rede (Controle Estático e *Anti-Spoofing*):** A primeira linha de defesa avalia a topologia e a origem dos dados. O firewall bloqueia tráfego lateral não autorizado (isolando clientes entre si) e descarta pacotes cujos IPs de origem não pertençam à sub-rede esperada, mitigando tentativas primárias de *IP Spoofing*.
- **Camada de Transporte (Validação de Sanidade de Protocolo):** Nesta etapa, o

sistema inspeciona os cabeçalhos TCP. Pacotes deformados (como *NULL Scans*, *FIN Scans* ou *XMAS Scans*) são bloqueados para impedir o mapeamento furtivo da rede. Além disso, pacotes SYN que carreguem *payloads* de forma anômala são descartados para evitar injeções prematuras de dados.

- **Camada de Aplicação:** Caso o pacote inicie uma conexão estruturalmente válida, seu conteúdo (*payload*) é inspecionado. O firewall busca por assinaturas específicas de ataques (como comandos maliciosos, ex: `rm -rf`), mitigando explorações de vulnerabilidades a nível de aplicação.
- **Análise Comportamental (Mitigação Volumétrica e *Rate Limiting*):** Para lidar com ataques onde os pacotes são perfeitamente válidos (como um *TCP SYN Flood*), o sistema rastreia o comportamento dos emissores utilizando o Endereço MAC (Camada 2). A lógica estabelece um limiar restrito de **15 requisições SYN dentro de uma janela de 2 segundos**.

Fluxo de Execução do Descarte Comportamental (Mitigação Volumétrica e *Rate Limiting*): Ao capturar um pacote, a camada de processamento verifica se o MAC de origem já consta em uma *blacklist* temporal. Se estiver limpo, atualiza-se o histórico de requisições. Caso o MAC viole o limiar de conexões, a função aborta o encaminhamento — não acionando a função `send()` —, gera um log de alerta no console e insere o invasor na *blacklist*, aplicando penalidades de bloqueio escalonadas (de 60 segundos a 24 horas) conforme a reincidência. Pacotes normais, que sobrevivem a todas as validações, são repassados ao destino original.

2.3 Eficácia do Firewall

- **Resultado: O método foi capaz de mitigar a anomalia?**
- **Conclusão: Discorra sobre como o processamento no "roteador virtual" impactou a latência da rede e a segurança do host final.**

R.2.3

Sim, o método implementado demonstrou eficácia na mitigação da anomalia volumétrica. A atuação conjunta do limitador de taxa (baseado no rastreamento de pacotes SYN por endereço MAC) e da validação de regras nas camadas de rede, transporte e aplicação interceptou a rajada maliciosa antes que ela alcançasse o destino. O ataque, que em condições normais gerou um pico massivo de requisições e consumo excessivo de largura de banda, foi isolado com precisão. O firewall bloqueou o tráfego ofensivo inserindo o agressor em uma lista negra temporal, permitindo simultaneamente que o tráfego benigno continuasse fluindo e preservando a disponibilidade do servidor.

Conclusão: Impacto na Latência e na Segurança A introdução do "roteador virtual" alterou a dinâmica da rede, estabelecendo um *trade-off* direto e esperado entre segurança e desempenho:

- **Segurança do Host Final:** O ganho em resiliência foi substancial. O servidor alvo foi completamente blindado contra ataques de exaustão de recursos. Sem essa camada intermediária, o ataque forçaria o sistema operacional do destino a alocar memória para milhares de conexões semi-abertas, resultando em indisponibilidade imediata (DoS). O roteador virtual absorveu o impacto primário, atuando como um escudo ativo que barrou ameaças antes do processamento na máquina de destino.

- **Latência da Rede:** O processamento no roteador virtual introduziu um *overhead* (sobrecarga) inerente à sua arquitetura de desenvolvimento. Como o roteamento e a filtragem foram realizados via software em espaço de usuário (*user-space*) utilizando a biblioteca Scapy, cada pacote exigiu decodificação completa, passagem por uma esteira de condicionais lógicas e buscas em memória (como checagem de *blacklist* e limiares). Essa inspeção aprofundada aumenta o tempo de ida e volta (RTT) de cada pacote e pode gerar *jitter* (variação de atraso). Embora o custo computacional em latência seja um limitador para redes de altíssimo rendimento em produção (onde técnicas de aceleração em hardware ou *kernel-space* seriam exigidas), no escopo desta implementação, a degradação moderada do desempenho provou-se um custo justificado para garantir a integridade absoluta e o funcionamento contínuo do ambiente.